Job 2 — Persistance via un service systemd

Compte rendu avec captures d'écran et réponses aux questions

Alexis Noiret · La Plateforme · Bachelor IT Cybersécurité · 27 octobre 2026

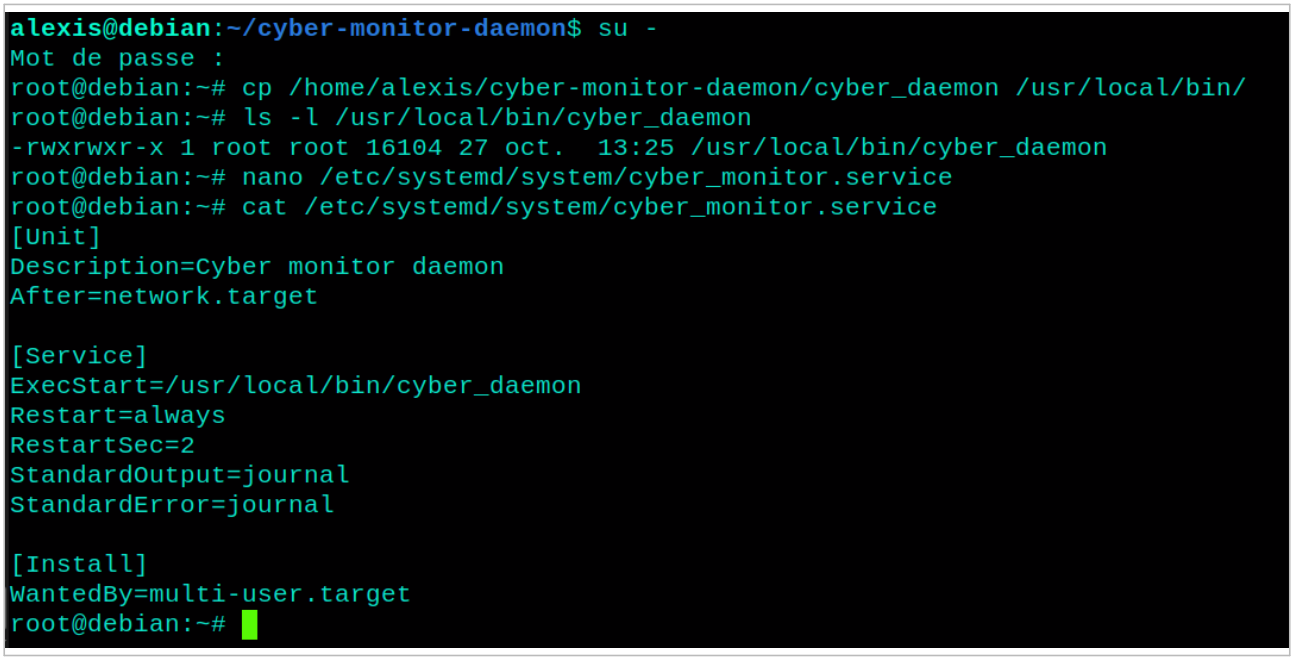
Environnement : Debian 13 (Trixie), systemd

Principe

Un programme lancé à la main s'arrête avec le terminal. Un **service systemd** est démarré et surveillé par le système : il démarre au boot et est relancé automatiquement s'il s'arrête. C'est cette propriété qui constitue la **persistance**.

1 — Installation du binaire et création de l'unité

```
su -
cp /home/alexis/cyber-monitor-daemon/cyber_daemon /usr/local/bin/
ls -l /usr/local/bin/cyber_daemon
nano /etc/systemd/system/cyber_monitor.service
```



Copie du binaire dans /usr/local/bin/ (propriétaire root, droits d'exécution) et contenu du fichier d'unité.

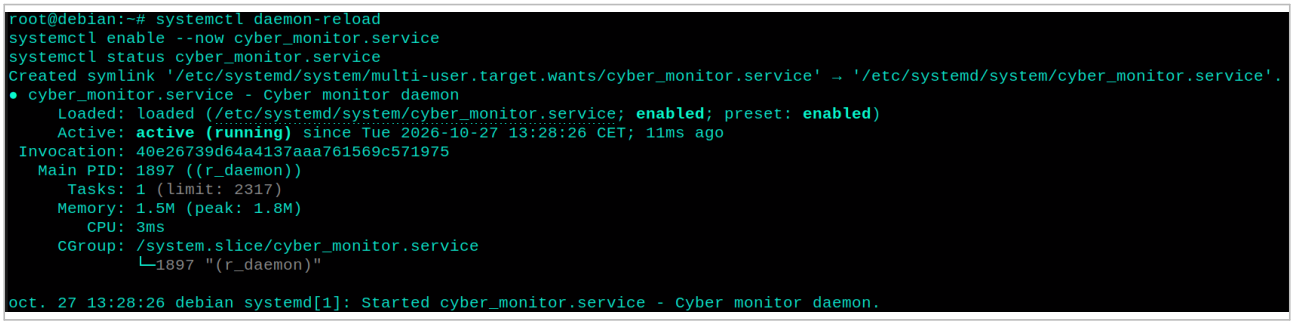
Cahier des charges du fichier d'unité

Le fichier `/etc/systemd/system/cyber_monitor.service` suit le format INI de systemd, avec trois sections :

- **[Unit]** : `Description=` donne un nom lisible au service ; `After=network.target` le fait démarrer après que le réseau est prêt (dépendance vis-à-vis du réseau).
- **[Service]** : `ExecStart=` est le chemin absolu du binaire ; `Restart=always` relance le programme dès qu'il s'arrête, quelle qu'en soit la raison (persistance) ; `RestartSec=2` attend 2 secondes avant de relancer ; `StandardOutput=journal` et `StandardError=journal` redirigent la sortie vers le journal système.
- **[Install]** : `WantedBy=multi-user.target` déclenche le démarrage automatique du service au boot (utilisé par systemctl enable).

2 — Chargement, activation et démarrage

```
systemctl daemon-reload
systemctl enable --now cyber_monitor.service
systemctl status cyber_monitor.service
```



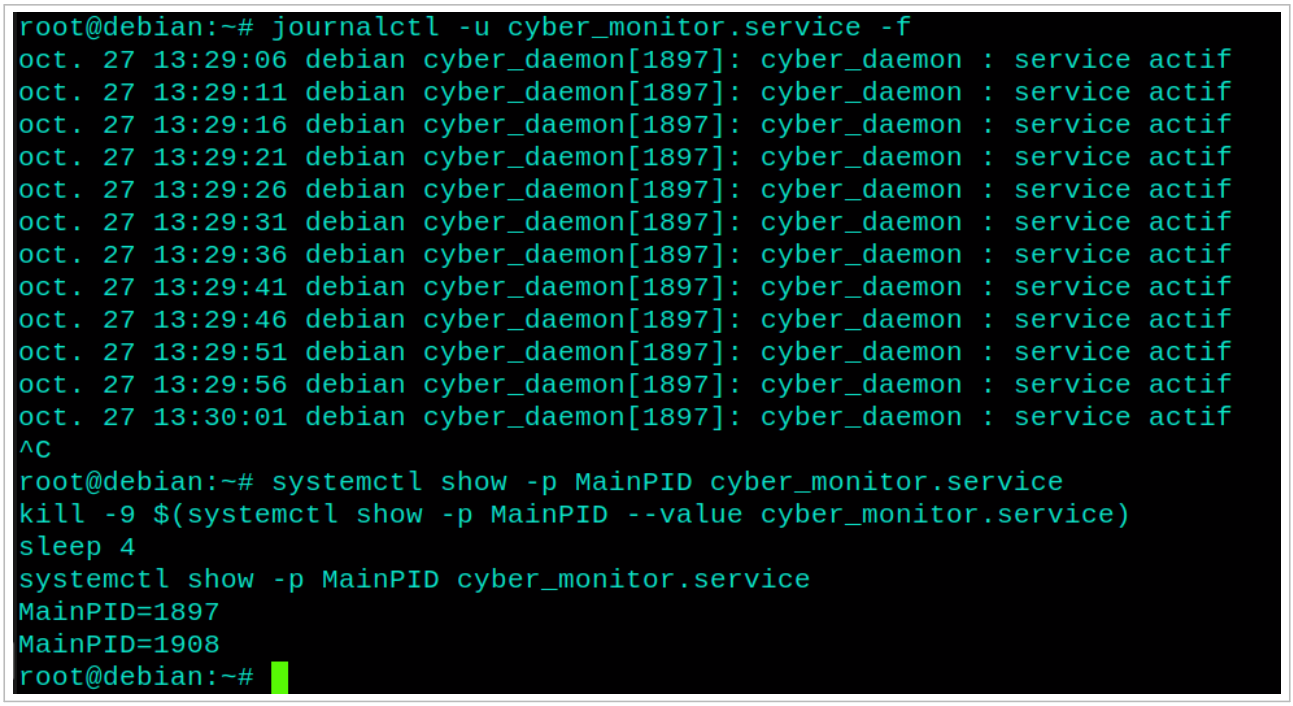
Le service est chargé (enabled) et actif (running).

- **daemon-reload** : demande à systemd de relire les fichiers d'unité pour découvrir le nouveau service.
- **enable** : crée le lien symbolique dans `multi-user.target.wants/` (visible dans la capture), ce qui active le démarrage au boot. **--now** démarre aussi le service immédiatement.
- **status** : `Loaded: ... enabled` confirme l'activation au boot, `Active: active (running)` confirme que le service tourne (PID 1897).

3 — Suivi des logs et test de persistance

```
journalctl -u cyber_monitor.service -f

systemctl show -p MainPID cyber_monitor.service
kill -9 $(systemctl show -p MainPID --value cyber_monitor.service)
sleep 4
systemctl show -p MainPID cyber_monitor.service
```



Logs du service en direct, puis test de persistance : le PID change après un kill -9.

Réflexion cyber / investigation : comment un analyste SOC détecte-t-il cette activité ?

Avec le gestionnaire de journaux centralisé de Linux, **journalctl**. La commande suivante filtre les logs de l'unité (**-u**) et les suit en direct (**-f**) :

```
journalctl -u cyber_monitor.service -f
```

La capture montre le message `cyber_daemon : service actif` toutes les 5 secondes (13:29:06, 13:29:11, 13:29:16...), avec le nom du programme et son PID entre crochets. Cela confirme aussi que `fflush(stdout)` fonctionne : sans lui, la sortie resterait dans le tampon et n'apparaîtrait pas dans le journal.

Un analyste peut aussi rechercher les unités récemment créées dans `/etc/systemd/system/` et lister les services activés avec `systemctl list-unit-files --state=enabled`.

Résultat du test de persistance

```
MainPID=1897   (avant le kill -9)
MainPID=1908   (après : systemd a relancé un nouveau processus)
```

Le processus a été tué de force, et systemd l'a relancé en 2 secondes avec un nouveau PID : `Restart=always` assure la persistance.

Lecture supply chain / MITRE ATT&CK

Cette mécanique correspond à la technique **T1543.002** (Create or Modify System Process: Systemd Service). Un service installé ainsi survit à un kill et revient à chaque démarrage de la machine, sans nouvelle action de l'utilisateur. Au Job 3, le même mécanisme sera déclenché automatiquement par le script postinst d'un paquet Debian, exécuté en root à l'installation.